

Accessing the API

When building applications with Claude, understanding the complete request lifecycle helps you make better architectural decisions and debug issues more effectively. Let's walk through what happens from the moment a user clicks "send" in your chat interface to when Claude's response appears on screen.

Video Link: [Access the API](#)

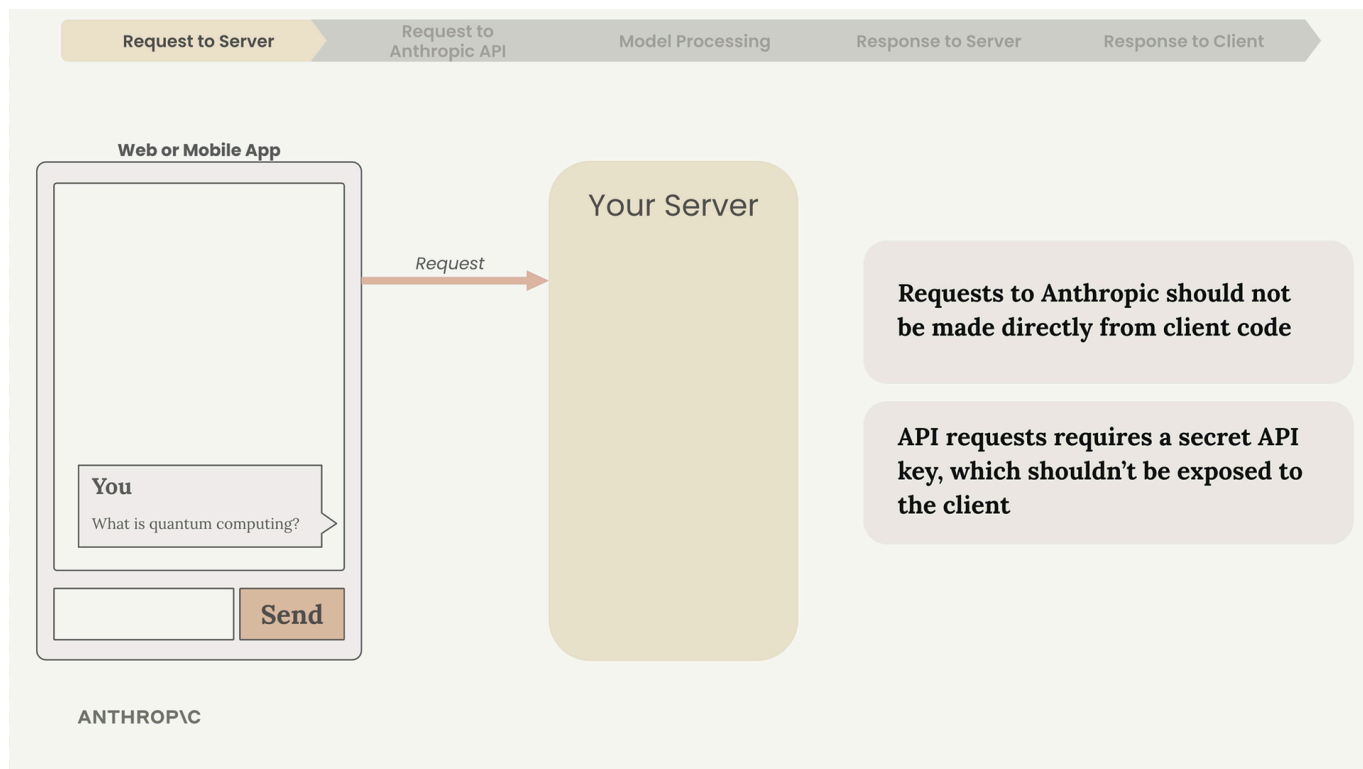
Chat example

The diagram illustrates a chat interface. It features a large rectangular area for messages. At the bottom, there is a text input field and a brown button labeled "Send". A message bubble is shown within the chat area, containing the text "You" and "What is quantum computing?".

ANTHROPIC

The Five-Step Request Flow

Every interaction with Claude follows a predictable pattern with five distinct phases: request to server, request to Anthropic API, model processing, response to server, and response to client.



Why You Need a Server

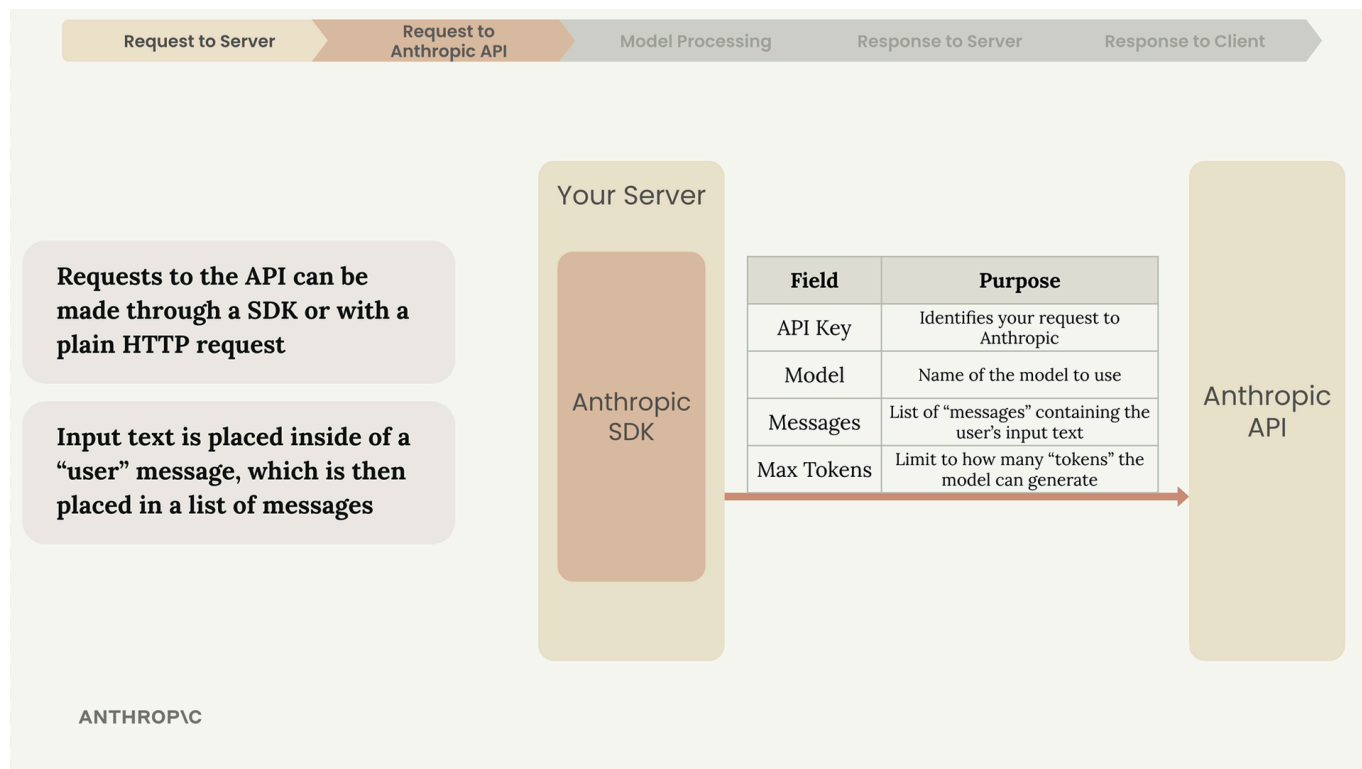
You should never make requests to the Anthropic API directly from client-side code. Here's why:

- API requests require a secret API key for authentication
- Exposing this key in client code creates a serious security vulnerability
- Anyone could extract the key and make unauthorized requests

Instead, your web or mobile app sends requests to your own server, which then communicates with the Anthropic API using the securely stored key.

Making API Requests

When your server contacts the Anthropic API, you can use either an official SDK or make plain HTTP requests. Anthropic provides SDKs for Python, TypeScript, JavaScript, Go, and Ruby.

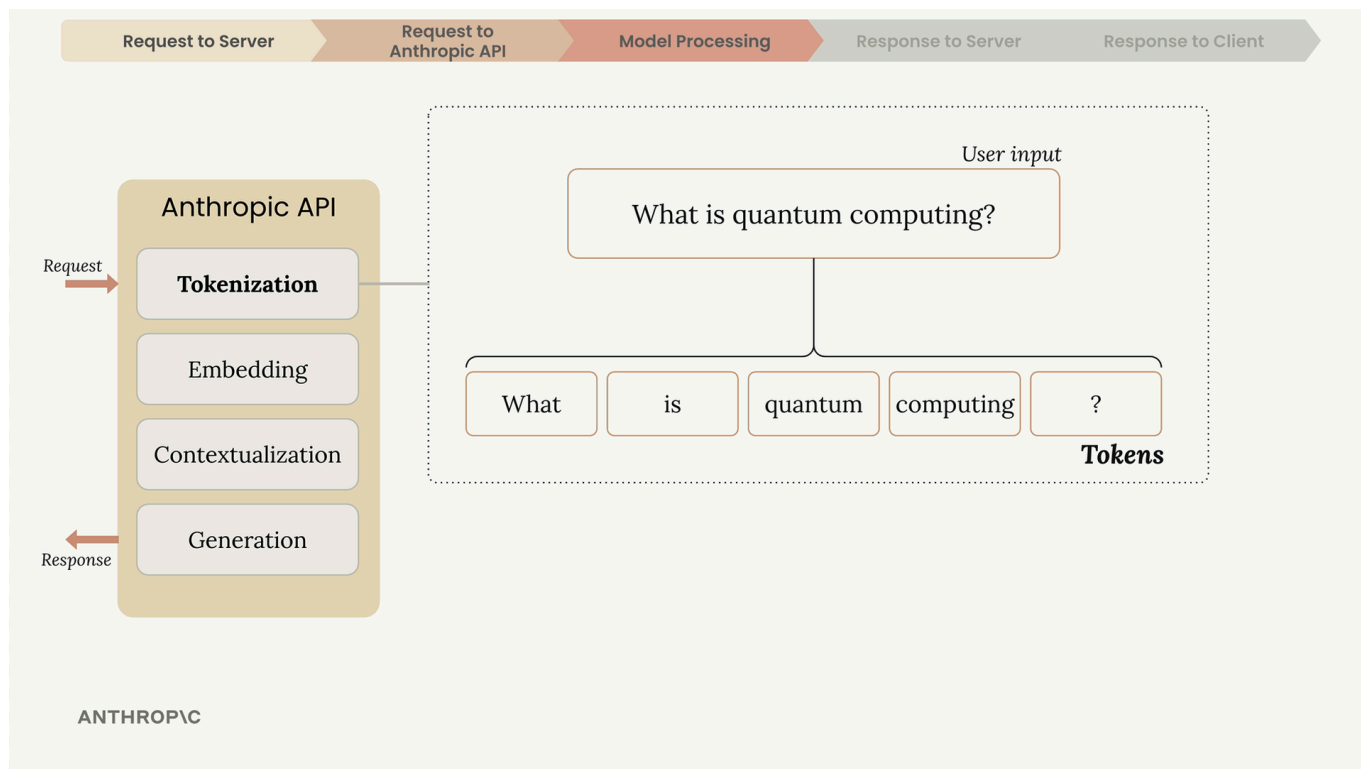


Every request must include these essential fields:

- **API Key** - Identifies your request to Anthropic
- **Model** - Name of the model to use (like "claude-3-sonnet")
- **Messages** - List containing the user's input text
- **Max Tokens** - Limit for how many tokens Claude can generate

Inside Claude's Processing

Once Anthropic receives your request, Claude processes it through four main stages: tokenization, embedding, contextualization, and generation.

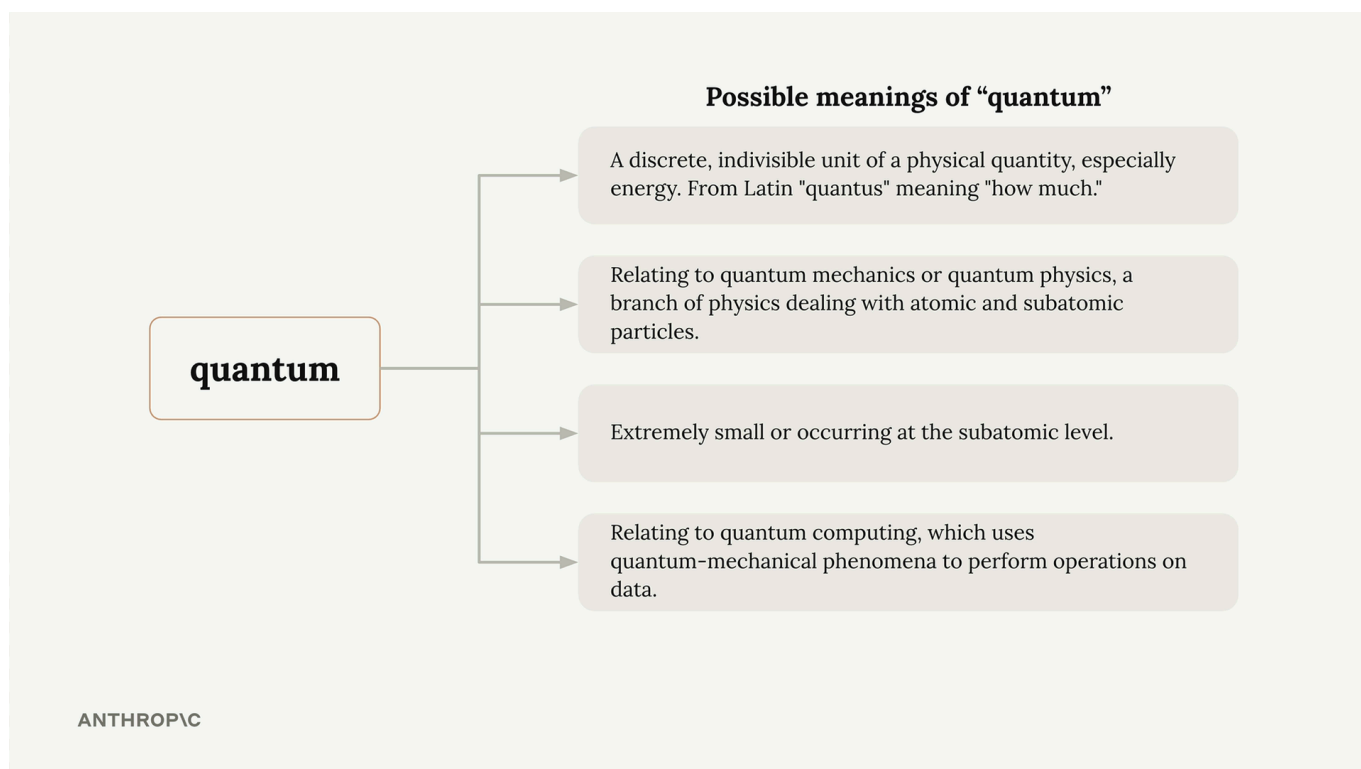


Tokenization

Claude first breaks your input text into smaller chunks called tokens. These can be whole words, parts of words, spaces, or symbols. For simplicity, think of each word as one token.

Embedding

Each token gets converted into an embedding - a long list of numbers that represents all possible meanings of that word. Think of embeddings as numerical definitions that capture semantic relationships.

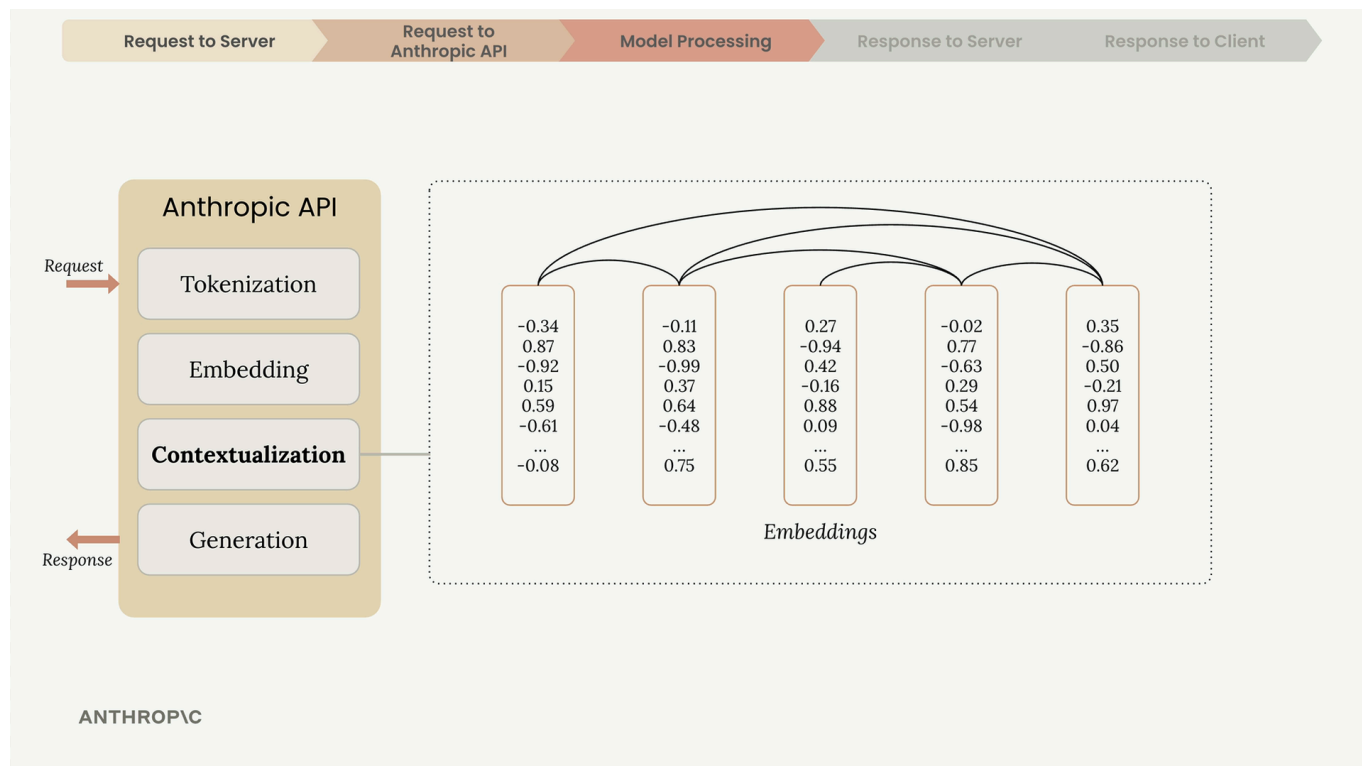


Words often have multiple meanings. For example, "quantum" could refer to:

- A discrete unit of physical quantity (physics)
- Quantum mechanics or quantum physics concepts
- Something extremely small or subatomic
- Quantum computing applications

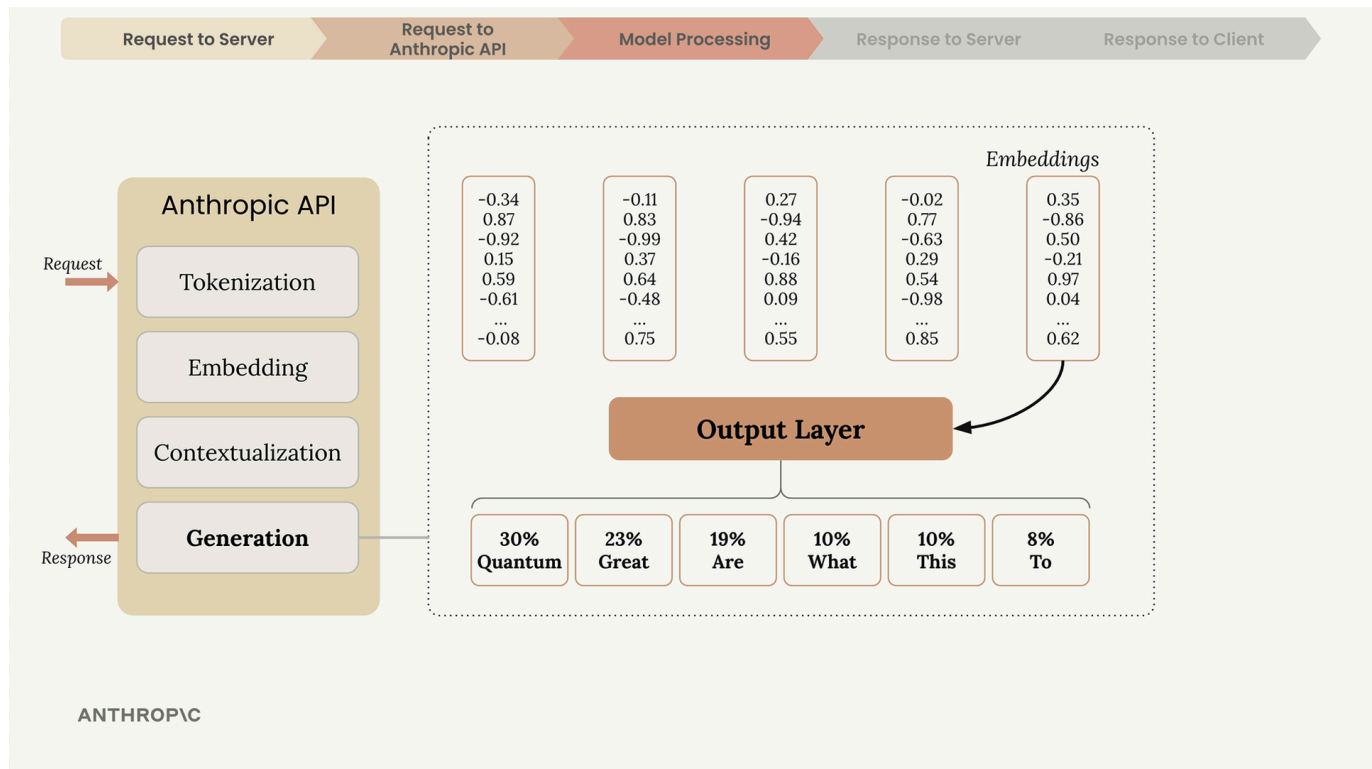
Contextualization

Claude refines each embedding based on surrounding words to determine the most likely meaning in context. This process adjusts the numerical representations to highlight the appropriate definition.



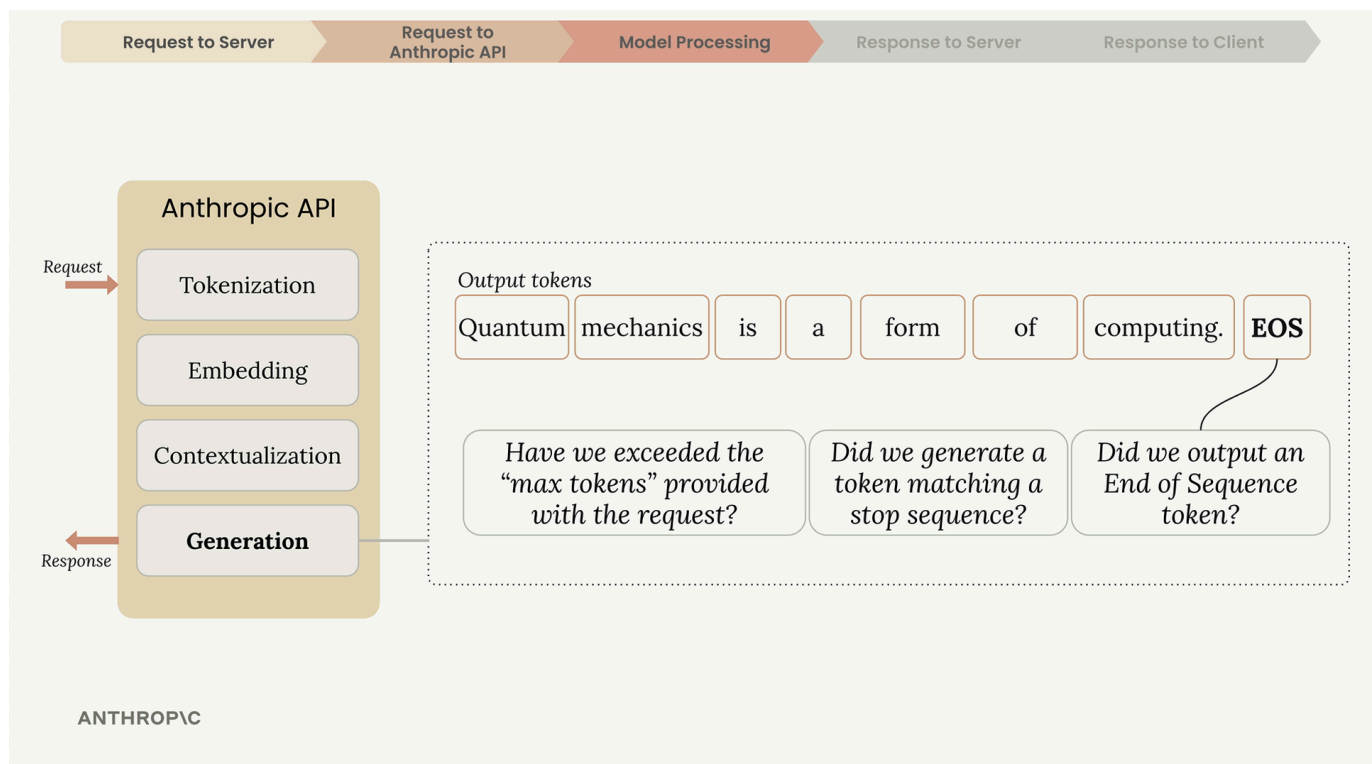
Generation

The contextualized embeddings pass through an output layer that calculates probabilities for each possible next word. Claude doesn't always pick the highest probability word - it uses a mix of probability and controlled randomness to create natural, varied responses.



When Claude Stops Generating

After each token, Claude checks several conditions to decide whether to continue:

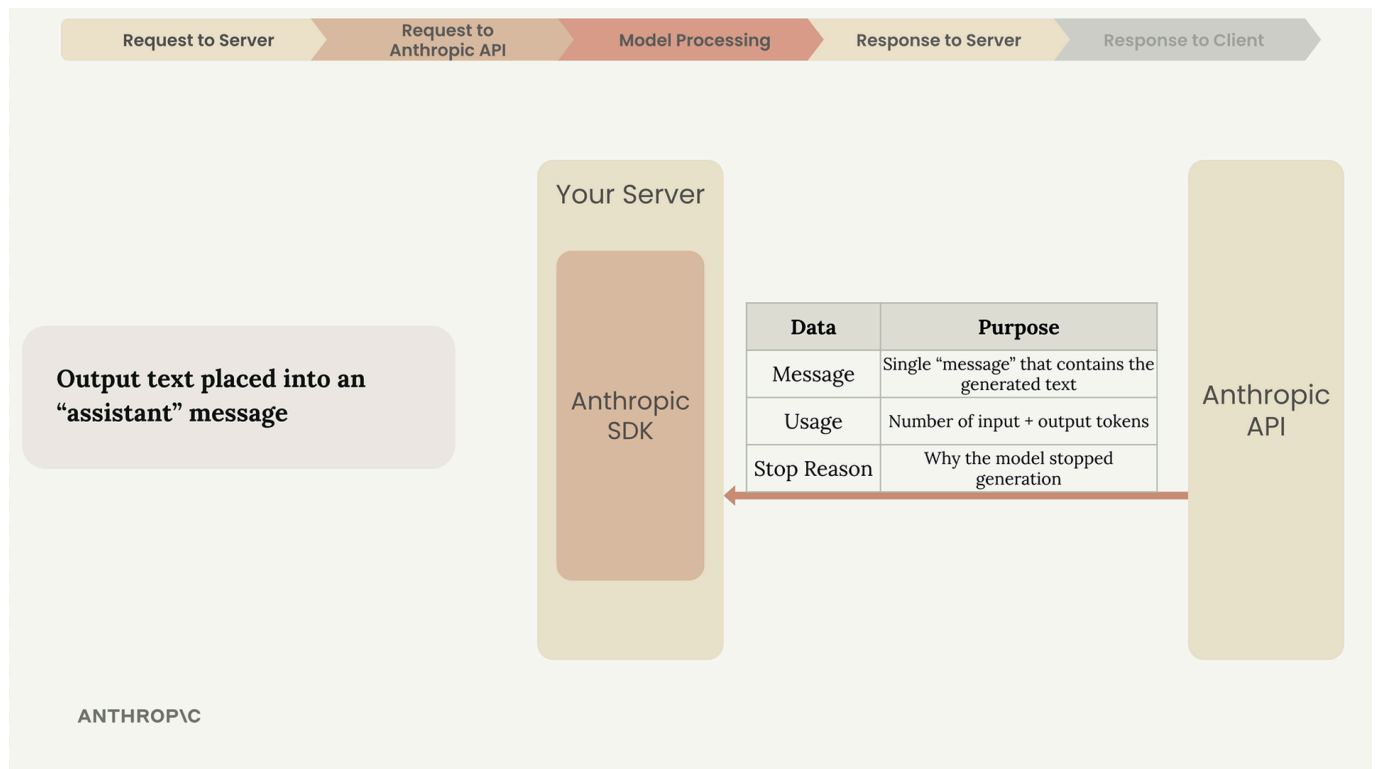


- **Max tokens reached** - Has it hit the limit you specified?
- **Natural ending** - Did it generate an end-of-sequence token?
- **Stop sequence** - Did it encounter a predefined stop phrase?

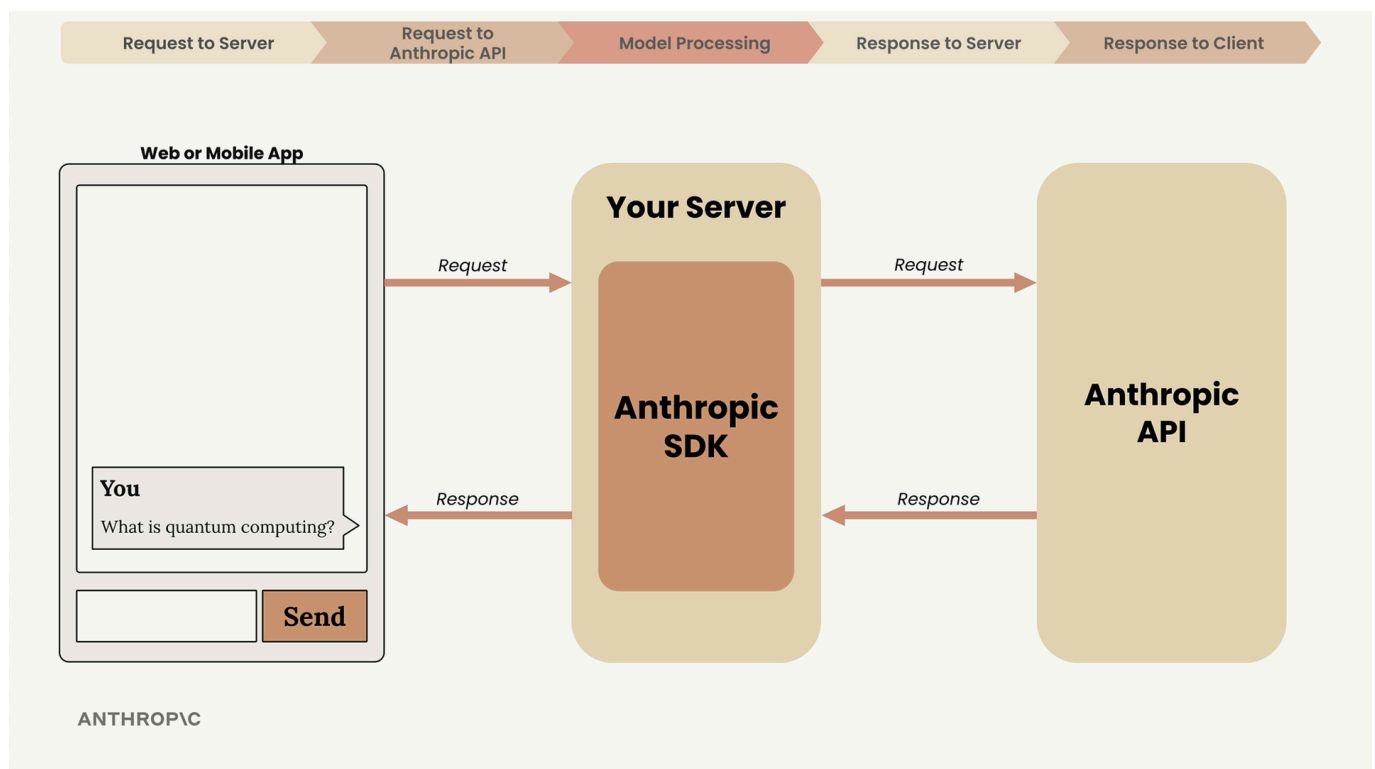
The API Response

When generation completes, the API sends back a structured response containing:

- **Message** - The generated text
- **Usage** - Count of input and output tokens
- **Stop Reason** - Why generation ended



Your server receives this response and forwards the generated text back to your client application, where it appears in the user interface.



Key Takeaways

Understanding this flow helps you:

- Design secure architectures that protect your API keys
- Set appropriate token limits for your use case
- Handle different stop reasons in your application logic
- Debug issues by understanding where they might occur in the pipeline

Don't worry about memorizing every detail - the goal is familiarizing yourself with the terminology and overall process you'll encounter when working with Claude's API.